SOFTWARE ENGINEERING MINI-PROJECT

Software Architecture and Design Specification

Task Manager Application — SAD

Document control	Value
Project	Task Manager Application v1.0
Version	1.0
Prepared / revised	15 October 2026
Status	Working baseline —approved
Technology baseline	Next.js (React), Node.js/Express REST API, MongoDB/Mongoose
Team	Akash Kiran Vernekar (PES1UG24AM338); Vismay H S (PES1UG24AM325); Basawaraj Panchal (PES1UG24AM350); Trisha G Shetty (PES1UG24AM308)
Submission deadline	30 October 2026

Control note: These documents define the proposed scope, architecture and verification plan. They do not claim the application is already implemented, deployed, tested, or approved. Update revision history, actual Jira keys, build status, test evidence and sign-offs as work is completed.

Document Control

Version	Date	Owner	Change
1.0	15 October 2026	Project team	New consistent design baseline linked to SRS v1.0 and STP v1.0; includes UML models, data design, API contracts, security and deployment/testing decisions.

Approval status: pending team and instructor review. This is an implementable target design; repository code and actual runtime behavior must be verified before describing it as implemented.

1. Introduction

1.1 Purpose and scope

This SAD translates the SRS Task Manager Application v1.0 into an architecture and implementation design. It covers Next.js/React UI, Express REST modules, MongoDB/Mongoose data access, main API contracts, UML component and sequence models, security, CI integration and traceability.

1.2 Related documents

SRS_TaskManagerApplication.docx v1.0 (requirements source); STP_TaskManagerApplication.docx v1.0 (verification plan). Any change to requirement IDs must be synchronized in all three.

2. Goals, Constraints and Decisions

Decision	Baseline / rationale
Architecture	Layered modular monolith: Next.js presentation; one Node.js/Express REST API; Mongoose data-access layer; MongoDB. Small team and course scope do not justify independently deployed microservices.
UI framework	Next.js with React components for routing/layout/rendering; UI calls the backend through typed/centralized API client. Choose the supported router and align implementation config at project scaffold.
API boundary	REST/JSON with explicit validation and centralized errors. Express is a separate process/service; Next.js is not the primary persistence layer.
Persistence	MongoDB with Mongoose schemas, validation, indexes and timestamps. API/service layer owns authorization and referential consistency.
Authentication	bcrypt password hashing; signed, expiring JWT. Token delivery/storage implementation must be selected securely during implementation; prefer HttpOnly/Secure/SameSite cookie for browser sessions if compatible with the API setup, and protect against CSRF as applicable.
Quality	Automated checks in GitHub Actions; Jira for requirements/work; feature branches and pull requests for integration.

3. Architecture

3.1 Logical Layers

Layer	Components / responsibility	Forbidden responsibility
Presentation	Next.js pages/layouts and reusable React UI components: auth screens, project views, task forms, board, comments, notifications, dashboard.	No direct MongoDB access; no sole reliance on client-side role checks.
API / Business	Express routes/controllers, middleware (auth, authorization, validation, rate limiting), services: Auth/User, Project/Task, Collaboration/Notification, Search/Reporting.	Do not duplicate business rules in UI only; do not return persistence secrets.
Data access	Mongoose models/repositories, query helpers, indexes, consistent mapping and reference checks.	No direct invocation from browser; avoid embedding route-specific policy in data models only.
Database	MongoDB User, Project, Task, Comment and Notification collections.	Never expose DB credentials to frontend; production data not used in automated tests.

3.2 Component Diagram (PlantUML source)

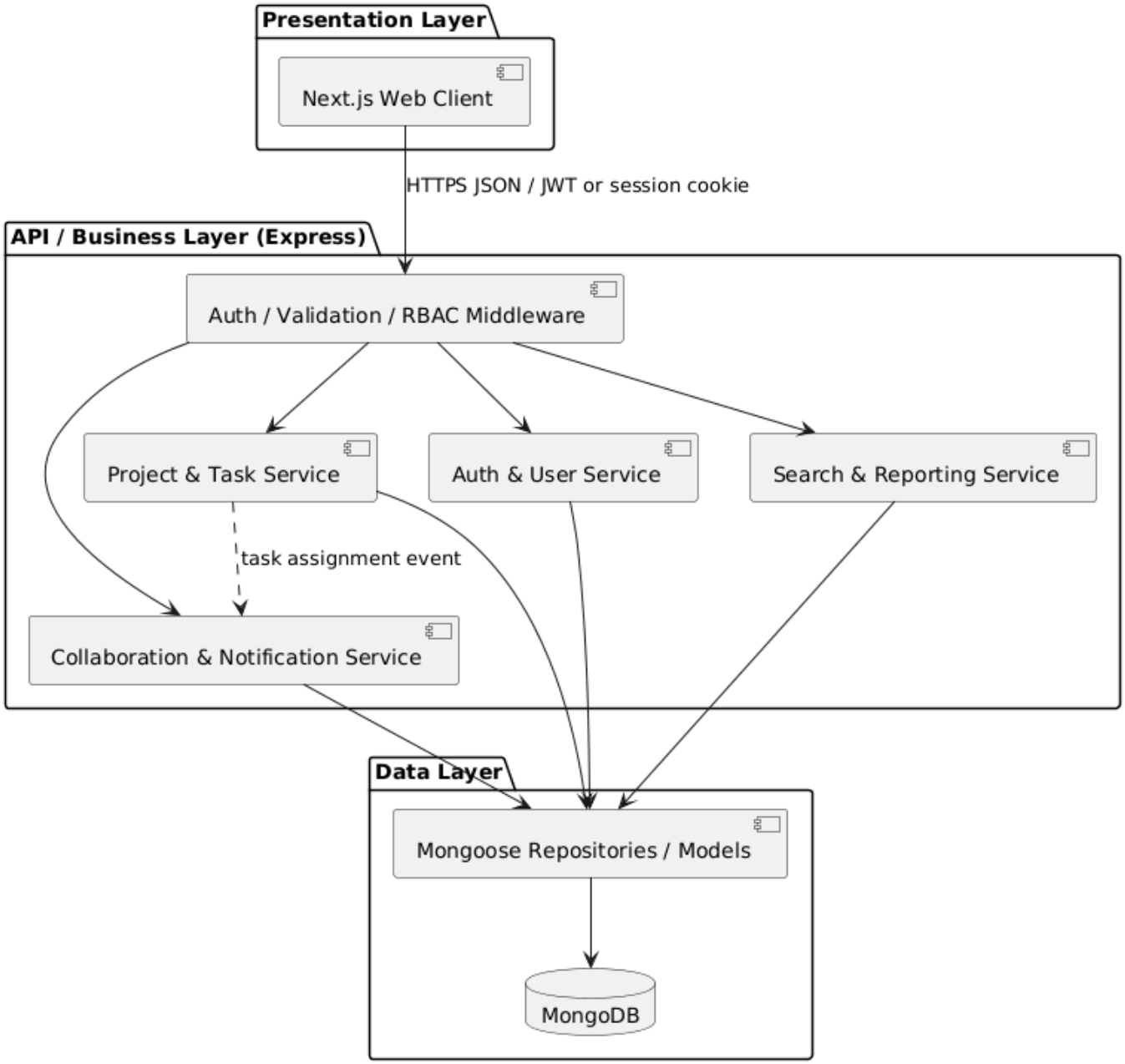


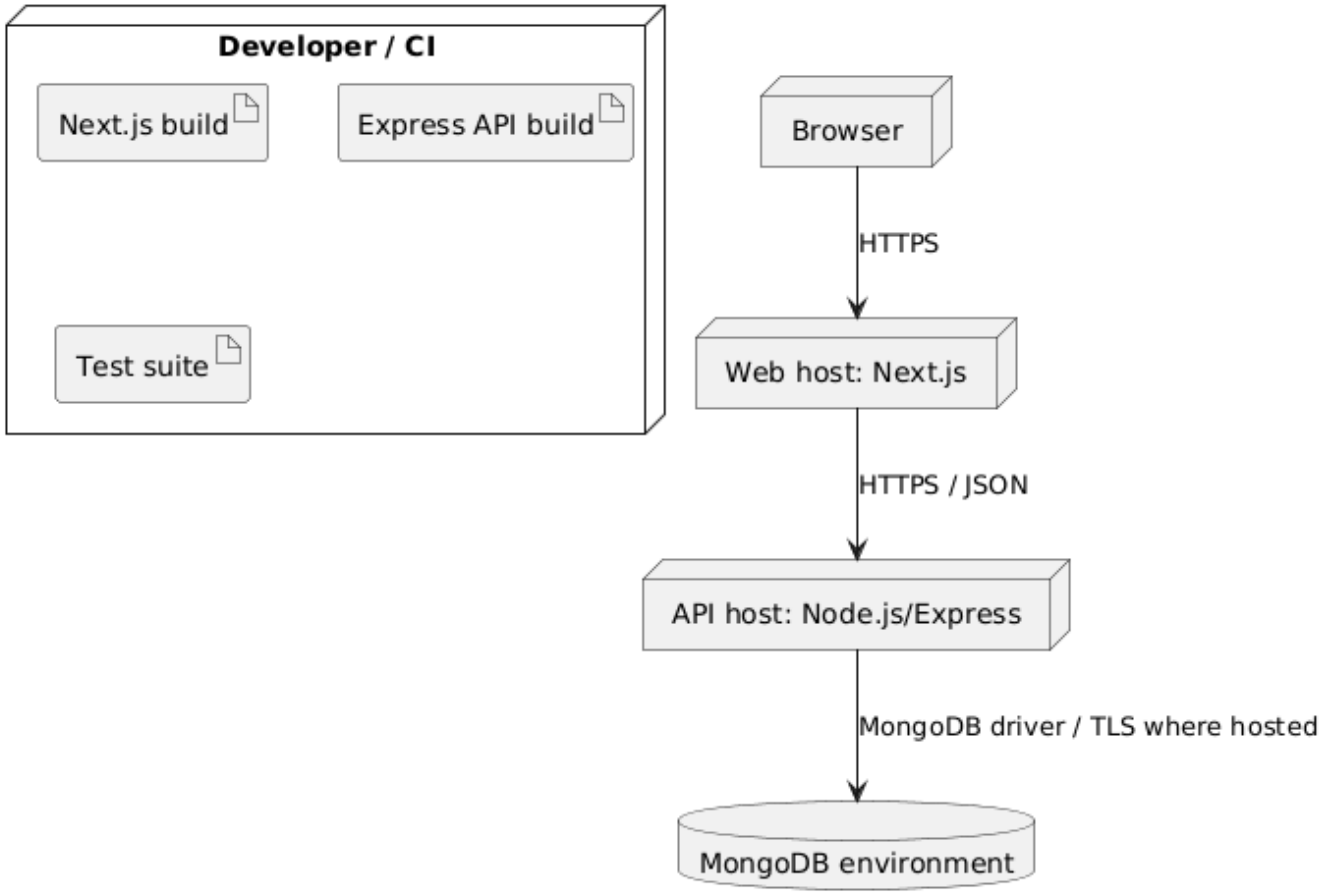
Diagram interpretation: the UI depends on the API, services depend on repository/model interfaces, and only server-side data access communicates with MongoDB. Assignment notifications are created after the task change has been validated and persisted. The PlantUML source can be rendered into a UML figure with PlantUML or pasted into a compatible diagram tool.

3.3 Component Responsibilities

Component	Responsibilities	SRS traceability
Next.js Web Client	Responsive routes, forms, board columns, task details/comments, search/filter, dashboard, notification display; accessible loading/error/empty states.	TM-F-001..042; TM-NF-004
Auth & User Service	Registration, uniqueness, login, bcrypt, token creation/validation support, profile changes.	TM-F-001..003; TM-NF-003; PRJ-SR-001/002
Project & Task Service	Project CRUD, membership changes, task CRUD, assignment and status transitions; membership validation.	TM-F-010..012; TM-F-020..023; PRJ-SR-003
Collaboration & Notification	Task comments, assignment notification creation, due-date reminder generation, notification read state and recipient isolation.	TM-F-030..032; PRJ-SR-003/005
Search & Reporting	Validated task query filters, scoped dashboard aggregates and safe CSV export.	TM-F-040..042; PRJ-SR-003/005
Mongoose/Data Access	Schemas, references, validation, indexes, repository/query functions and	SRS §10; TM-NF-001; TM-NF-006

Component	Responsibilities	SRS traceability
	timestamps.	
CI workflow	Install from lockfile, lint, automated tests and production build for API/UI; publish status to pull request.	TM-NF-006; STP §6

3.4 Deployment View



Use separate local/dev/test configuration. The CI workflow must use an isolated test database or a disposable service. Production secrets must be stored in the selected hosting/CI secret manager. Deployment provider and domain are implementation decisions not assumed here.

3.5 Data Model and Integrity

Collection	Fields / shape	Indexes and integrity
User	name, emailNormalized, passwordHash, role, createdAt, updatedAt	Unique index on emailNormalized. Schema enum for role. Never serialize passwordHash.
Project	name, description, ownerId, memberIds[], timestamps	Index ownerId/memberIds as query shape requires; owner is member/admin by explicit policy.
Task	title, description, status, priority, dueDate, projectId, assigneeIds[], timestamps	Indexes projectId/status; assigneeIds; dueDate/status where measured. Validate project membership of each assignee.
Comment	taskId, authorId, text, createdAt	Indexes taskId/createdAt. Validate task access before read/write.
Notification	userId, taskId, type, message, read, createdAt	Indexes userId/read/createdAt; recipient access checks; unique/event dedupe strategy for due reminders.

Data lifecycle decisions to finalize in Jira before implementation: what happens to comments/notifications when a project or task is deleted; how user removal updates project memberships and assignees; reminder timezone and duplicate suppression. The implementation must not leave dangling references that break list/report queries.

4. API Design

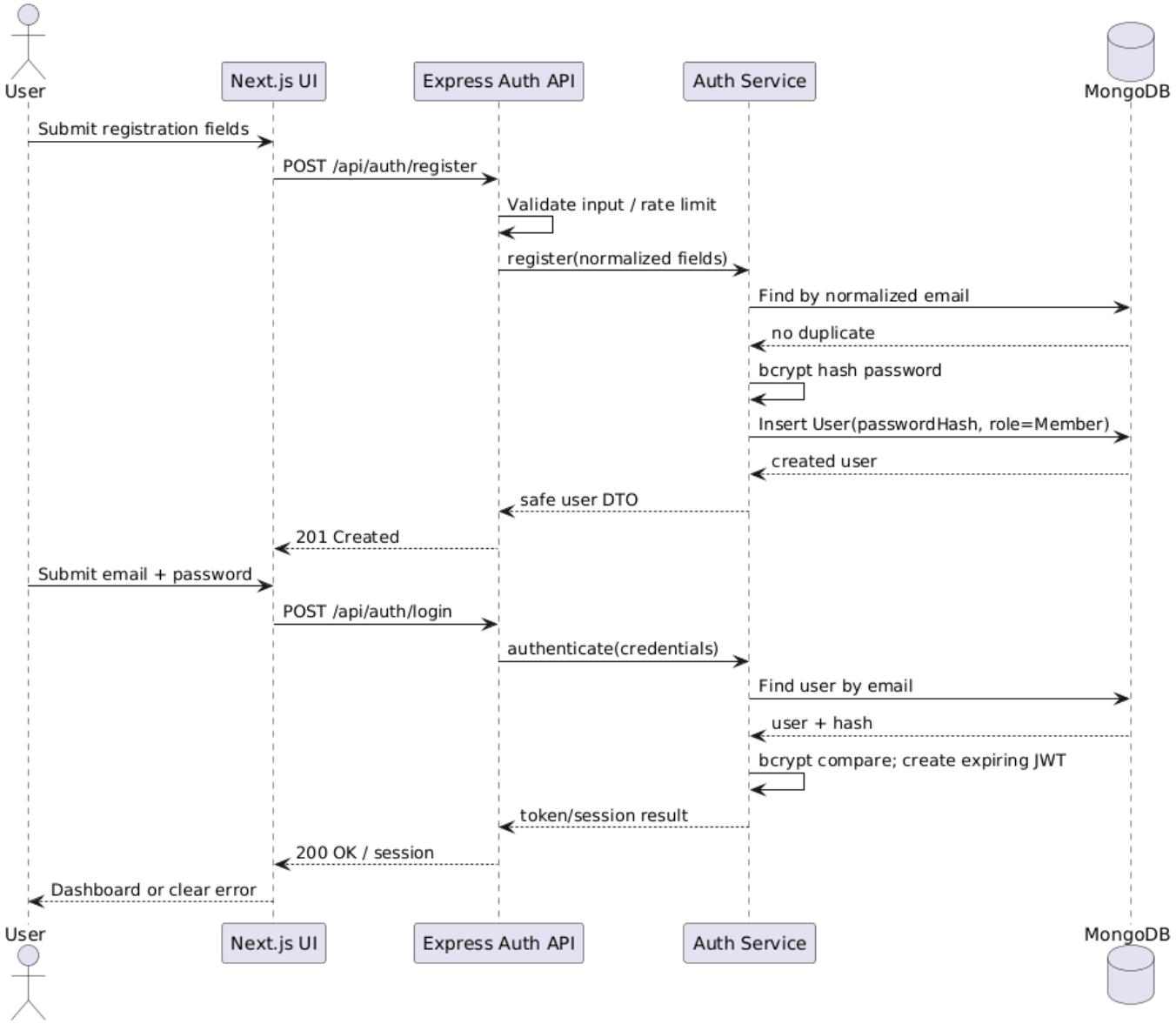
API base path: /api. Protected paths require authentication and server-side authorization. Request bodies are validated before service execution. Errors follow {errorCode, message, timestamp}; stack traces and sensitive values are not sent to clients.

Method / path	Purpose	Success	Typical errors / validation
POST /api/auth/register	Create account	201 {userId,email}	400 invalid fields/password; 409 duplicate email; role never self-elevated.
POST /api/auth/login	Authenticate	200 {token,expiresIn} or cookie/session equivalent	401 bad credentials; 429 rate limit.
GET /api/auth/me	Current session user	200 safe user profile	401 missing/invalid/expired session.
POST /api/projects	Create project	201 project DTO	400 invalid name; 401 unauthenticated; 403 insufficient role.
POST /api/projects/{id}/members	Add member	200/201 member list	400 invalid payload; 403 unauthorized manager; 404 project/user.
POST /api/projects/{id}/tasks	Create task/assign	201 task DTO	400 validation; 403 forbidden/nonmember; 404 project.
PATCH /api/tasks/{id}	Edit task fields	200 task DTO	400 validation; 403 unauthorized; 404 task.
PATCH /api/tasks/{id}/status	Update status	200 {taskId,status}	400 invalid status; 401 invalid session; 403 unauthorized; 404 task.
POST /api/tasks/{id}/comments	Add comment	201 comment DTO	400 empty/too long; 403 non-member; 404 task.
GET /api/notifications	List own notifications	200 paginated list	401 session invalid; always scoped by current user.
GET /api/projects/{id}/summary	Status counts / overdue	200 summary DTO	403 unauthorized; 404 project.
GET /api/projects/{id}/tasks/export.csv	CSV export	200 text/csv	403 unauthorized; safe quoting and formula-prefix protection.

4.1 Validation and Error Contract

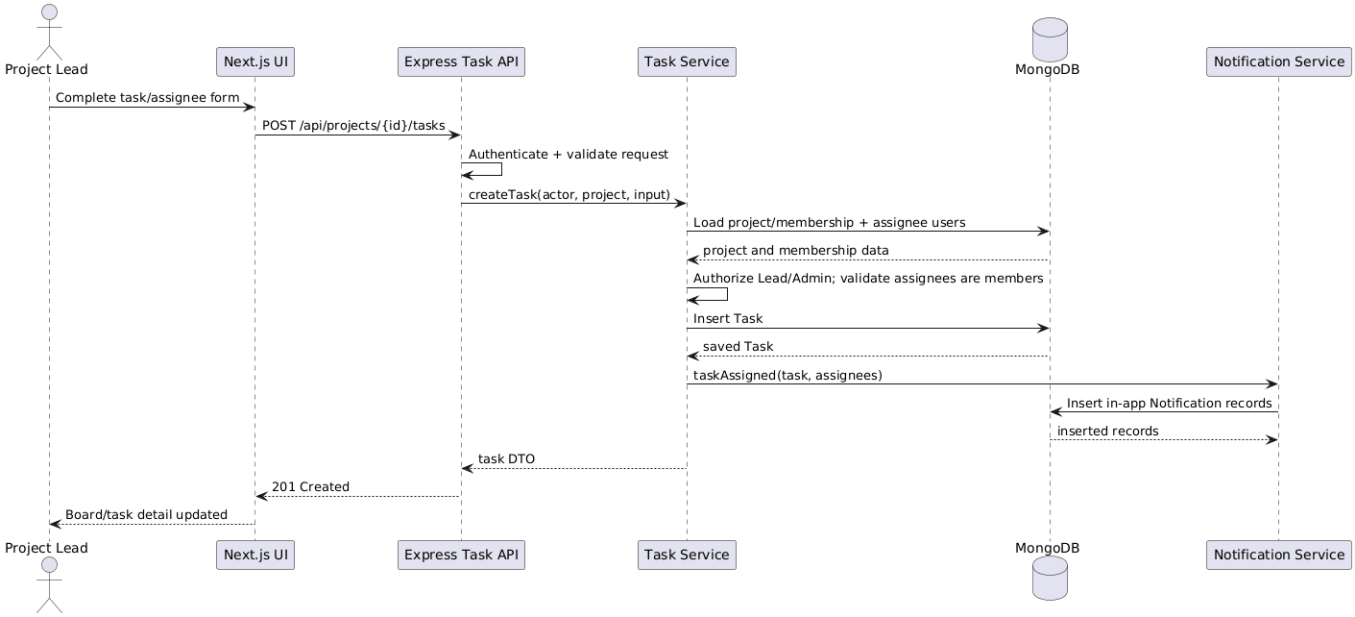
- Validate IDs, enums, dates, string lengths, arrays and required fields at the request boundary; trim/normalize email; reject invalid dates.
- Never use client-supplied userId/role as trusted identity. Derive actor from validated session and verify membership in service logic.
- Use 400 for invalid request, 401 for unauthenticated, 403 for authenticated but forbidden, 404 for missing resource, 409 for conflict/duplicate, 429 for rate limit and 500 only for unexpected server failure.
- Pagination should be supported for task/comment/notification lists if data volume could be large; ordering must be stable.
- CSV export must quote separators/newlines and prevent spreadsheet formula injection.

4.2 Sequence Diagram — Register and Login



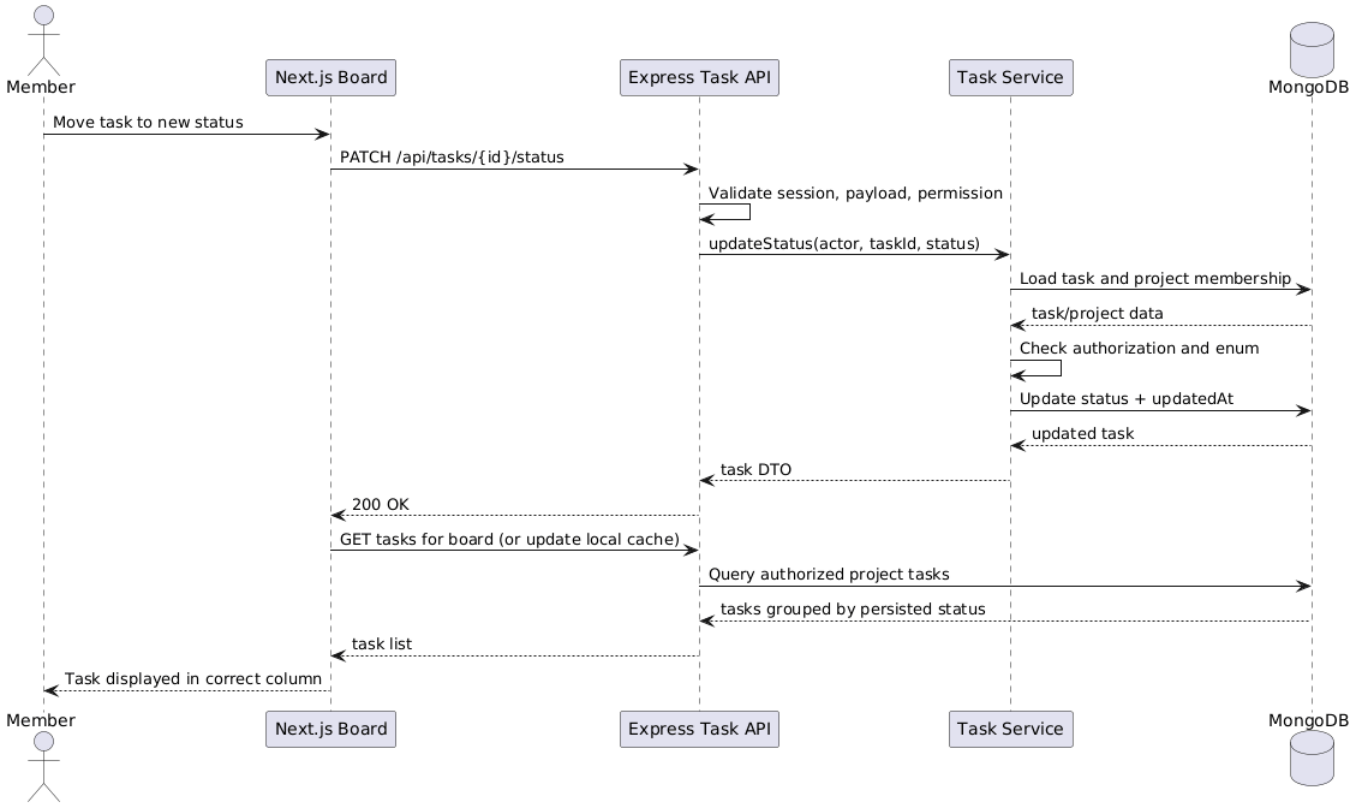
Traceability: TM-F-001, TM-F-002, TM-F-003, PRJ-SR-001, PRJ-SR-002, PRJ-SR-006; tests TC-AUTH-01/02/03 and TC-SEC-01/02/06.

4.3 Sequence Diagram — Create and Assign Task



Traceability: TM-F-010, TM-F-020, TM-F-021, TM-F-022, TM-F-030, PRJ-SR-003; tests TC-TASK-01, TC-PROJ-01/02, TC-BOARD-01, TC-NOTIF-01, TC-SEC-03.

4.4 Sequence Diagram — Status Update and Board Refresh



Traceability: TM-F-012, TM-F-023, PRJ-SR-003; tests TC-TASK-03, TC-BOARD-02, TC-E2E-01.

5. Security Architecture and Threats

Threat (STRIDE)	Control	Verification
Spoofing	bcrypt password hash; expiry-limited signed JWT/session; login rate limits.	TC-SEC-01/02/06
Tampering	Validate payloads; write only through API/service; server-side enums and references; code review.	TC-SEC-05, API integration tests
Repudiation	createdAt/updatedAt, authorId on comments and task creator/update attribution where required; avoid sensitive log data.	Data assertions and logging review
Information disclosure	HTTPS; scoped queries; never return passwordHash; own-notification scope; safe errors.	TC-SEC-01/03/04
Denial of service	Auth rate limit; request/body size limits; list pagination; bounded expensive queries.	TC-SEC-06; load-test report
Elevation of privilege	RBAC plus project membership checks on every API read/write/export.	TC-SEC-03 and negative access matrix

6. Cross-Cutting Design

6.1 Logging and Monitoring

- Structured logs include timestamp, level, request/correlation ID, route and safe actor ID; no passwords, tokens, connection strings or full sensitive body.
- Central error middleware maps expected validation/authorization/not-found/conflict failures to stable codes and catches unexpected errors.
- Measure API latency (p50/p95), request/error rate and observed availability in the environment actually used; report test duration and profile for load results.

6.2 Frontend Design

- Group UI by feature: auth, projects, tasks/board, comments/notifications, reporting and common components.
- Use centralized API client for base URL, auth/session behavior, error mapping and response types.
- Provide loading, validation, network-error and no-results states; keyboard access and contrast checked by STP.
- Do not trust client-side route protection as authorization; API must independently enforce it.

6.3 CI/CD and Branching

- Pull request validation: lockfile-based dependency install, lint, unit tests, API tests, production builds. Add E2E smoke tests as soon as a repeatable test environment exists.
- Branch model: main is protected by team convention; feature/fix/docs branches; merge via reviewed pull requests linked to Jira.
- Secrets in GitHub Actions/environment settings, not files committed to Git. CI uses synthetic data and separate DB URI.
- A green CI run means checks passed for that commit; it does not by itself imply security sign-off, uptime proof or 100-user performance pass.

7. Requirements and Test Traceability

SRS requirement group	Design modules	STP validation
TM-F-001..003	Next.js Auth UI, Auth/User Service, User model	TC-AUTH-01..03; TC-SEC-01/02/06
TM-F-010..012	Task UI, Task Service, Task model	TC-TASK-01..03
TM-F-020..023	Project/Task Service, Project membership, board UI	TC-PROJ-01/02; TC-BOARD-01/02; TC-E2E-01
TM-F-030..032	Collaboration/Notification Service, Comment and Notification models	TC-NOTIF-01/02; TC-COMM-01
TM-F-040..042	Search/Reporting Service, scoped query aggregation and CSV serializer	TC-SEARCH-01; TC-REPORT-01/02
TM-NF-001/002/005	API/data layer, logging/metrics and query indexes	TC-PERF-01; OPS-MON-01; TC-PERF-02

SRS requirement group	Design modules	STP validation
TM-NF-003/004/006; PRJ-SR-001..006	Auth middleware, UI accessibility, validation, CI controls	TC-SEC-01..06; TC-UX-01; TC-CI-01

8. Risks, Open Decisions and Next Steps

Risk / decision	Mitigation / owner action
Next.js vs separate Express boundary	Keep Express as the authoritative API and database access boundary; decide deployment topology in initial technical spike.
Role/authorization leakage	Create an endpoint-by-role access matrix and test both positive and negative access for every resource family.
Flexible MongoDB schema drift	Use Mongoose validation and review schema changes through PRs; add migration/backfill instructions for incompatible field changes.
Build/test environment instability	Use lockfile, Node version pin, synthetic fixtures and reproducible CI; run E2E in stable isolated environment.
Performance target beyond local capacity	Run repeatable k6/JMeter profile when staging is stable; document limits and deviations rather than invent results.
Reminder timing and data deletion policy	Record timezone, reminder window/dedupe, and deletion/cascade policy as Jira decisions before implementation.

Appendix A. API Error Envelope Example

```
{
  "errorCode": "VALIDATION_ERROR",
  "message": "One or more fields are invalid.",
  "timestamp": "2026-10-15T10:00:00.000Z"
}
```

Appendix B. Reference

Internal baseline documents: SRS_TaskManagerApplication.docx v1.0 and STP_TaskManagerApplication.docx v1.0. UML code is included in this document for repeatable rendering. Update revision and approvals when the team formally baselines the design.